

The background of the slide features a large, faint, light-blue circular logo of Tianjin University in the upper right corner. The logo contains the university's name in English ('TIANJIN UNIVERSITY') and Chinese ('天津大学'), along with the founding year '1895'. In the lower left corner, there is a faint, light-blue line drawing of a traditional Chinese building with a tiled roof and multiple windows.

内存管理

Memory Management



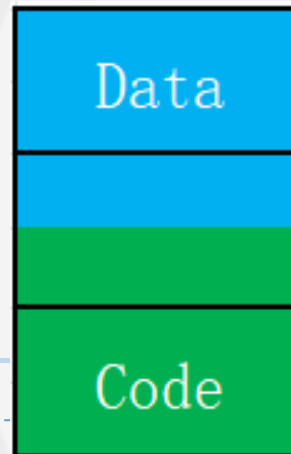
主要内容

- 分页式的缺点
- 分段式内存管理
- 段页式内存管理
- 各种内存管理方式比较

- 内存管理方案
 - 单一连续分配
 - 分区管理
 - 页式管理
 - 段式管理
 - 段页式管理



分页式的缺点



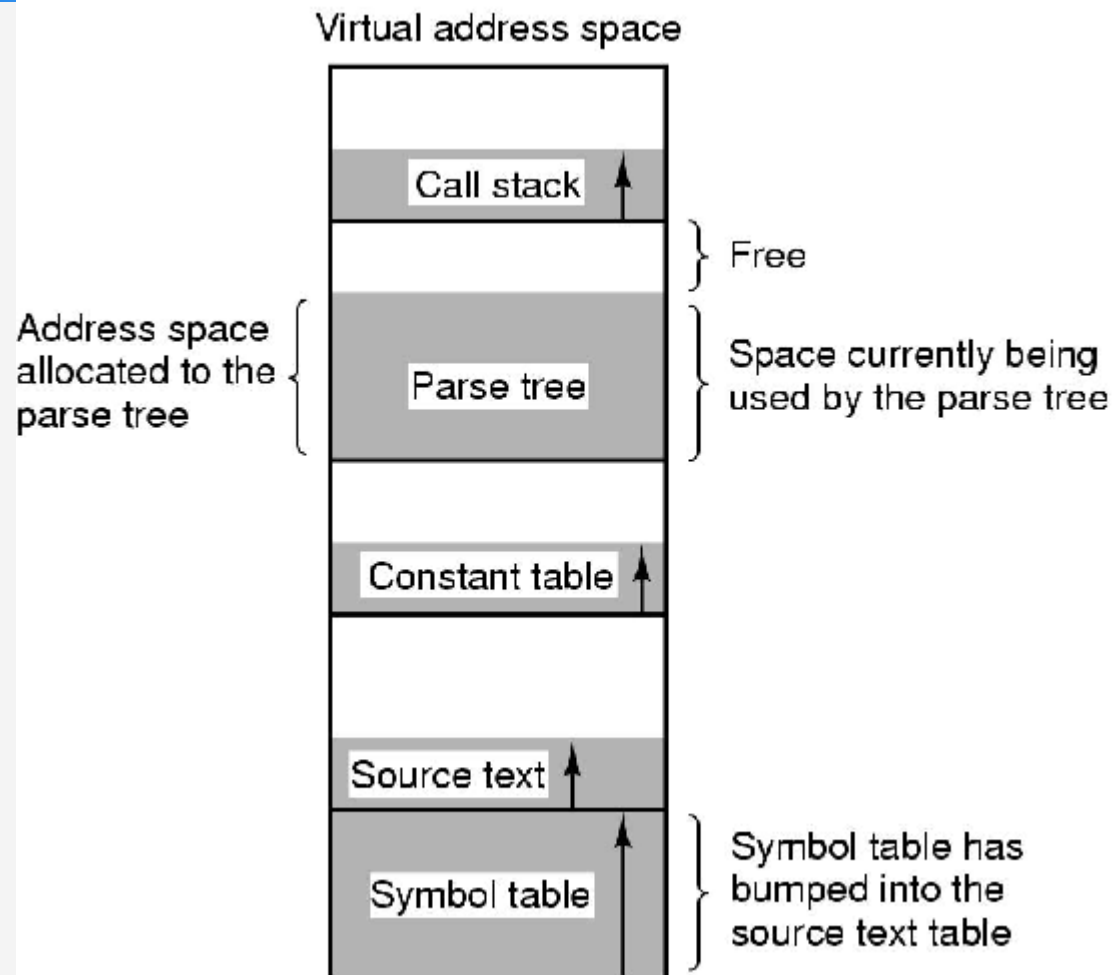
```
while(1) {  
    printf("hello");  
    printf("world");  
}
```

- 分页式内存管理有很多优点，如按需调入节约内存，不连续存放更灵活等。
- 但是，由于页面大小是固定的，分页式也存在一些问题：
 - 1. 可能在同一页中，混合了不同类型或不相关的内容。例如：页面大小4KB，代码6KB，数据6KB，如果连续编址，将占用三个页面，中间的页面就会混编代码和数据。代码的访问权限一般为可读不可写可执行，而数据的访问权限一般为可读可写不可执行，这给权限保护带来很多麻烦，类似的问题还有一个页面内既包含要在进程之间共享的代码，也包含私有代码，则整个页面的共享就无法实现等。
 - 2. 相关内容被分到不同的页面，例如一个循环的循环体代码被分到二个页面中。如果内存紧张，页面淘汰算法选择不当，可能导致相关内容的页面没有同步换入内存。极端情况下只有一个空闲页帧，二个页面甚至互相抢占同一页帧轮流换入，产生大量缺页中断，引发**内存抖动（Thrashing）**。



分段 (Segmentation)

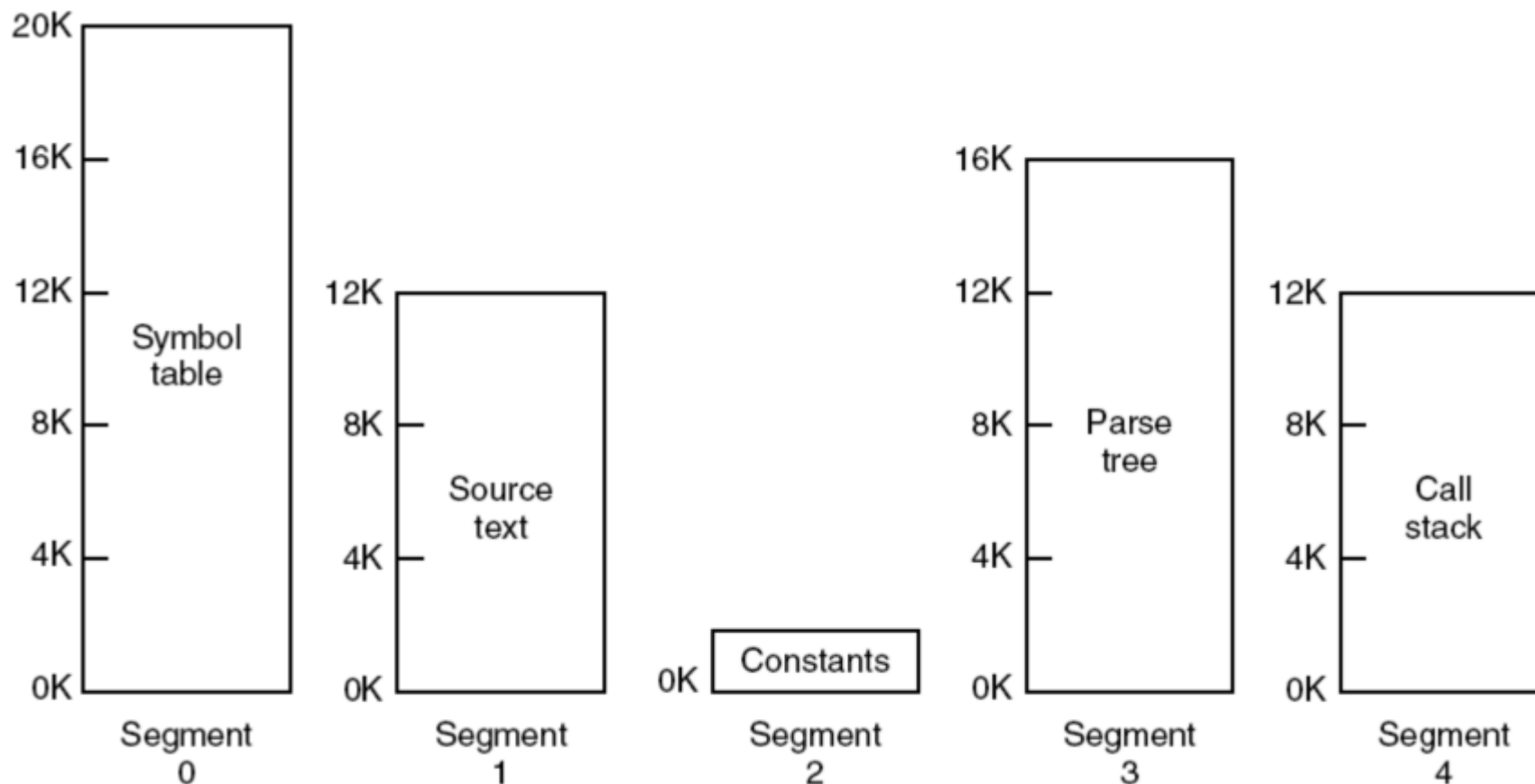
- 编译器在进行编译时会建立许多表，可能包括：
 - 正在为打印列表保存源文本（在批处理系统中）。
 - 符号表——变量的名称和属性。
 - 包含所使用的整数和浮点常量的表。
 - 语法树，程序的句法分析。
 - 用于编译器中过程调用的堆栈。
- 在一维地址空间中，随着表的增加，一个表可能会撞到另一个表。





分段式内存管理 (Segmentation)

- 分段内存允许每个表独立于其他表进行增长或缩小。





分段式内存管理 (Segmentation)

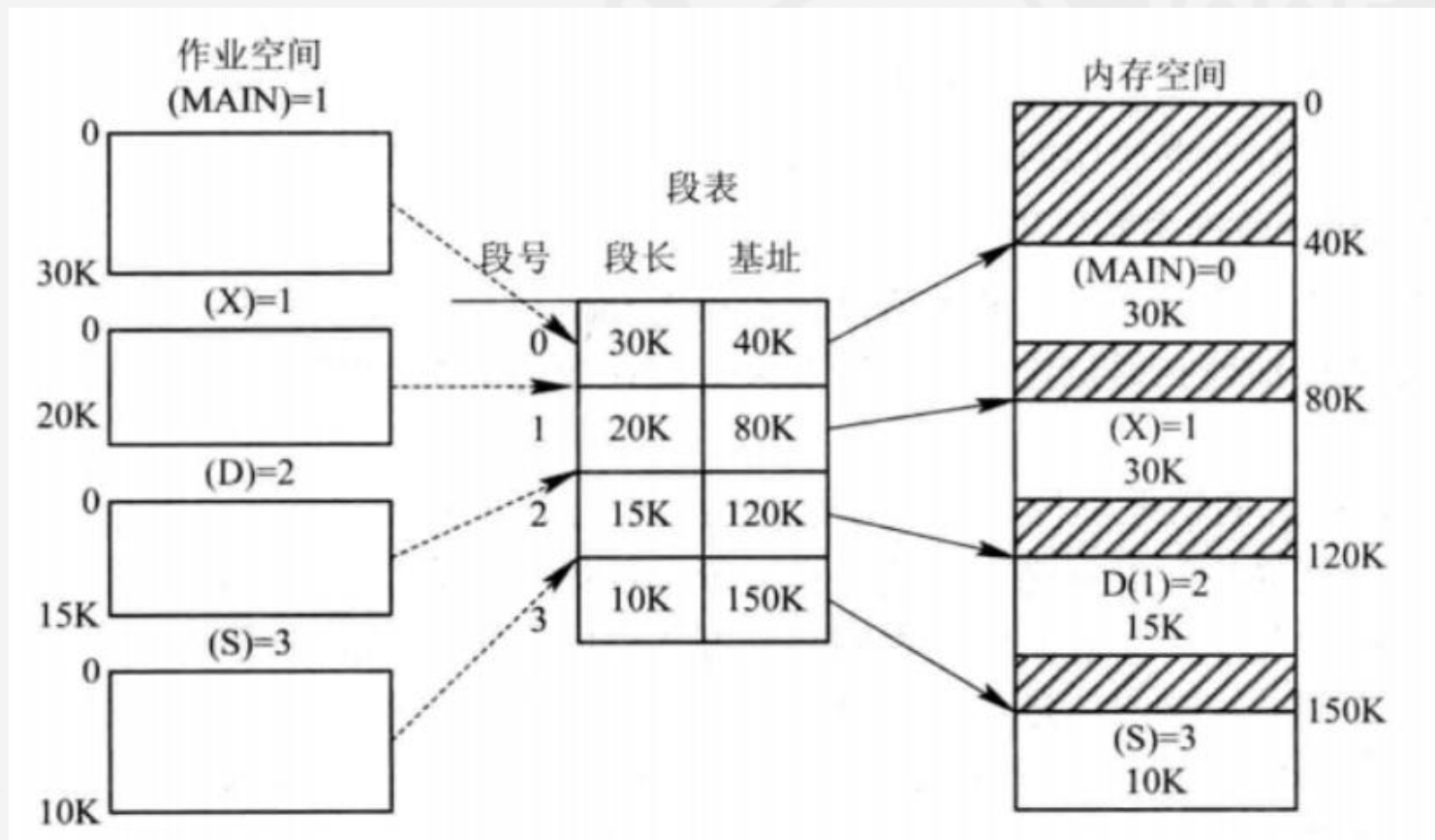
- 引入分段存储管理方式的原因：
 - 通常的程序都可以分为若干个段，每个段大多都是一个相对独立的逻辑单位。
 - 实现和满足信息共享，信息保护，动态链接以及信息的动态增长等需求，也都是以段为基本单位的。
- 在分段存储管理方式中，作业的地址空间被划分为若干个段，每个段定义了一组逻辑信息。例如，有主程序段MAIN、子程序段X、数据段D及栈段S等，通常，用段号代替段名。每个段都从0开始编址，并采用一段连续的地址空间。
- 段的长度由相应的逻辑信息组的长度决定，因此各段的长度并不相等。
- 分段式中的逻辑地址由段号和段内偏移地址组成。

段号	段内地址
----	------



段表 (Segment Table)

- 类似与分页系统，分段系统中段也是离散的分布在内存中，所以需也要为每个进程建立一个段映射表，简称段表。
- 每个段在表中占有一个表项，其中记录了该段在内存空间的起始地址（基址）和段的长度。

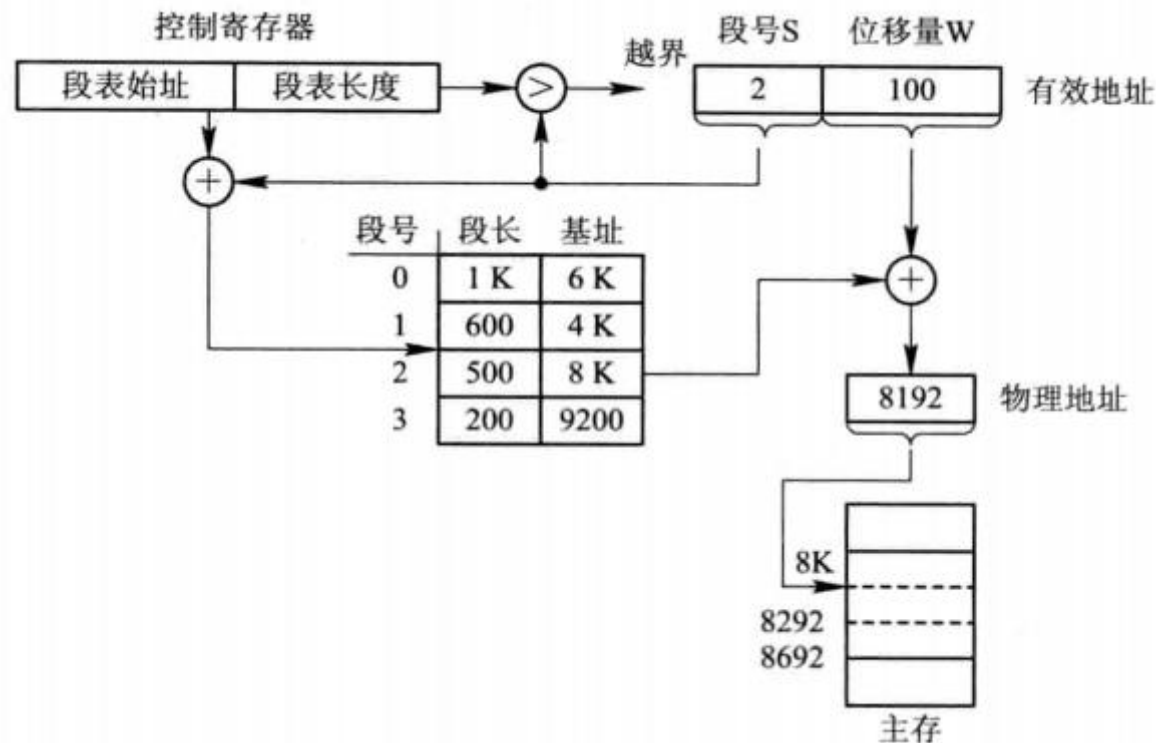




段表 (Segment Table)

为了实现进程从逻辑地址到物理地址的变换功能，在系统中设置了段表寄存器，用于存放段表始址和段表长度TL。在进行地址变换时：

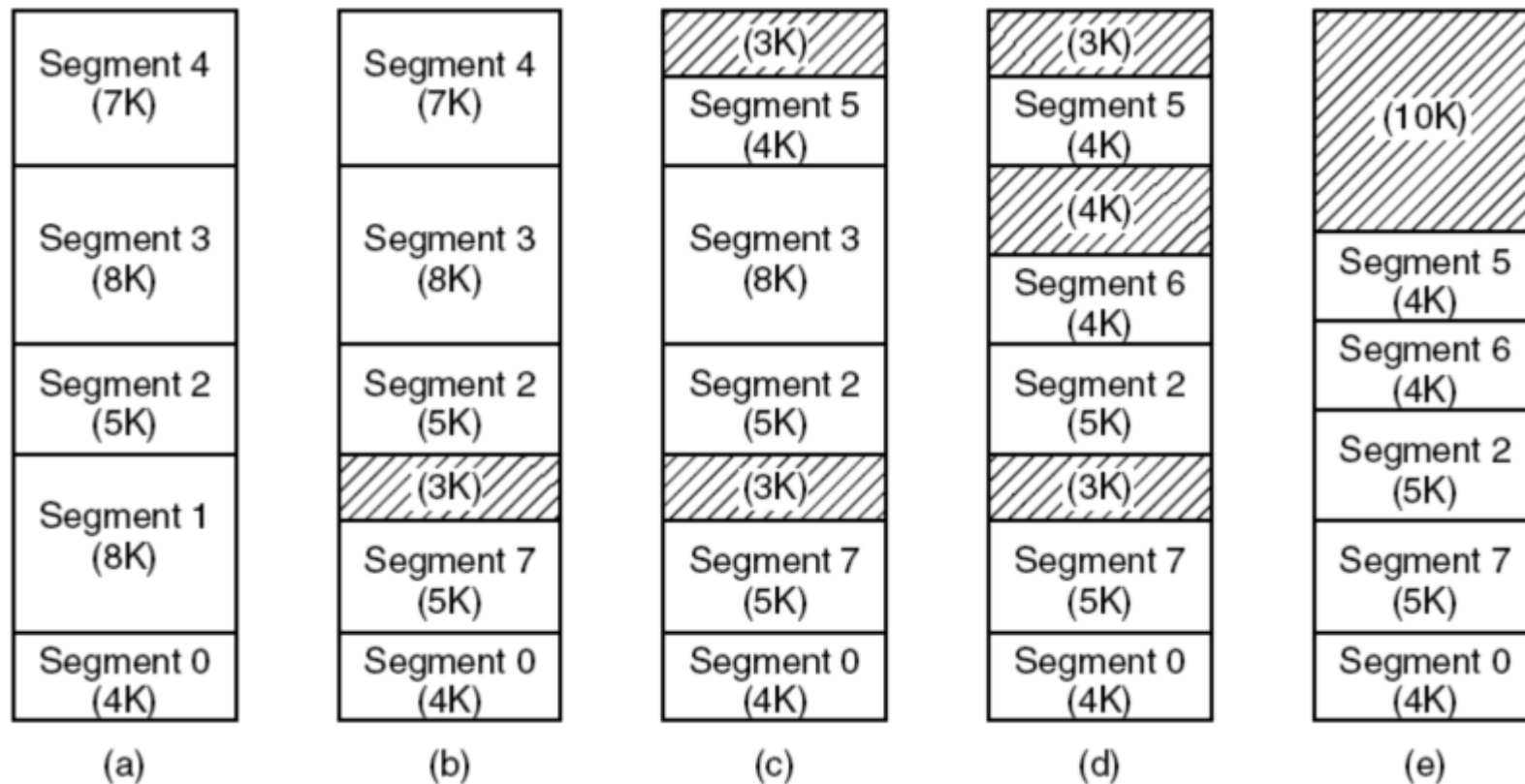
1. 系统将逻辑地址中的段号与段表长度TL进行比较。若 $S > TL$ ，表示段号太大，是访问越界，于是产生越界中断信号。
2. 若未越界，则根据段表的始址和该段的段号，计算出该段对应段表项的位置，从中读出该段在内存的起始地址。
3. 再检查段内地址d是否超过该段的段长SL。若超过，即 $d > SL$ ，同样发出越界中断信号。
4. 若未越界，则将该段的基址d与段内地址相加，即可得到要访问的内存物理地址。





分段 (Segmentation)

- 请求式分段也可以实现虚拟内存。操作系统只按需加载用到的段，将暂时用不到的段换出内存。
- 请求式分段在交换段时与可变分区中交换整个进程时，会面临类似的问题，如外部碎片和内存紧缩。





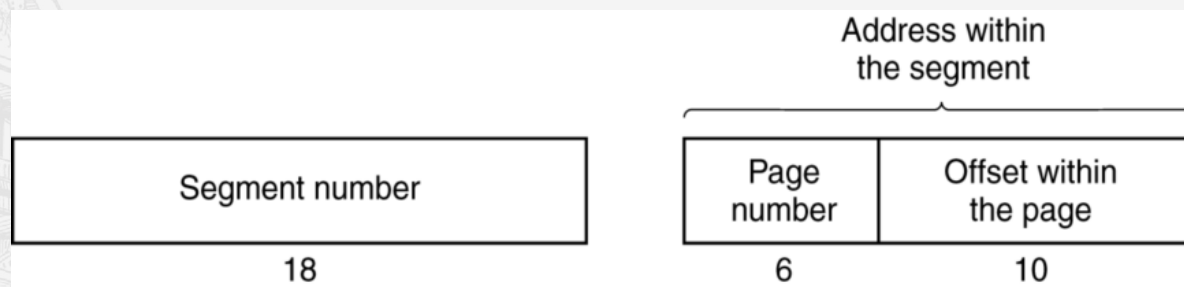
分段与分页对比

- 相似点：两者都采用了离散分配方式，都是通过地址映射机构来实现地址变化。区别点如下：
 - 页是信息的物理单位，分页仅仅是为了系统管理的需求，完全是系统的行为，对用户是不可见的。段是信息的逻辑单位，它通常是一组意义完整的信息，目的是更好的满足用户的需求。
 - 页的大小固定且由系统决定。而段的大小不定。分页的用户程序地址空间是一维的。而分段系统中，用户的地址空间是二维的。
 - 分页系统中换入换出的单元是页，分段系统中换入换出的单元是段。
- 分段系统的一个突出优点，是易于实现段的共享，即允许若干个进程共享一个或者多个分段。
- 分段还可以有效解决页面抖动（Thrashing）问题，因为相关的内容总是一起被换入换出的。
- 分段存在的问题与可变分区类似，即存在外部碎片，不便于内存空间的管理。



段页式存储管理方式

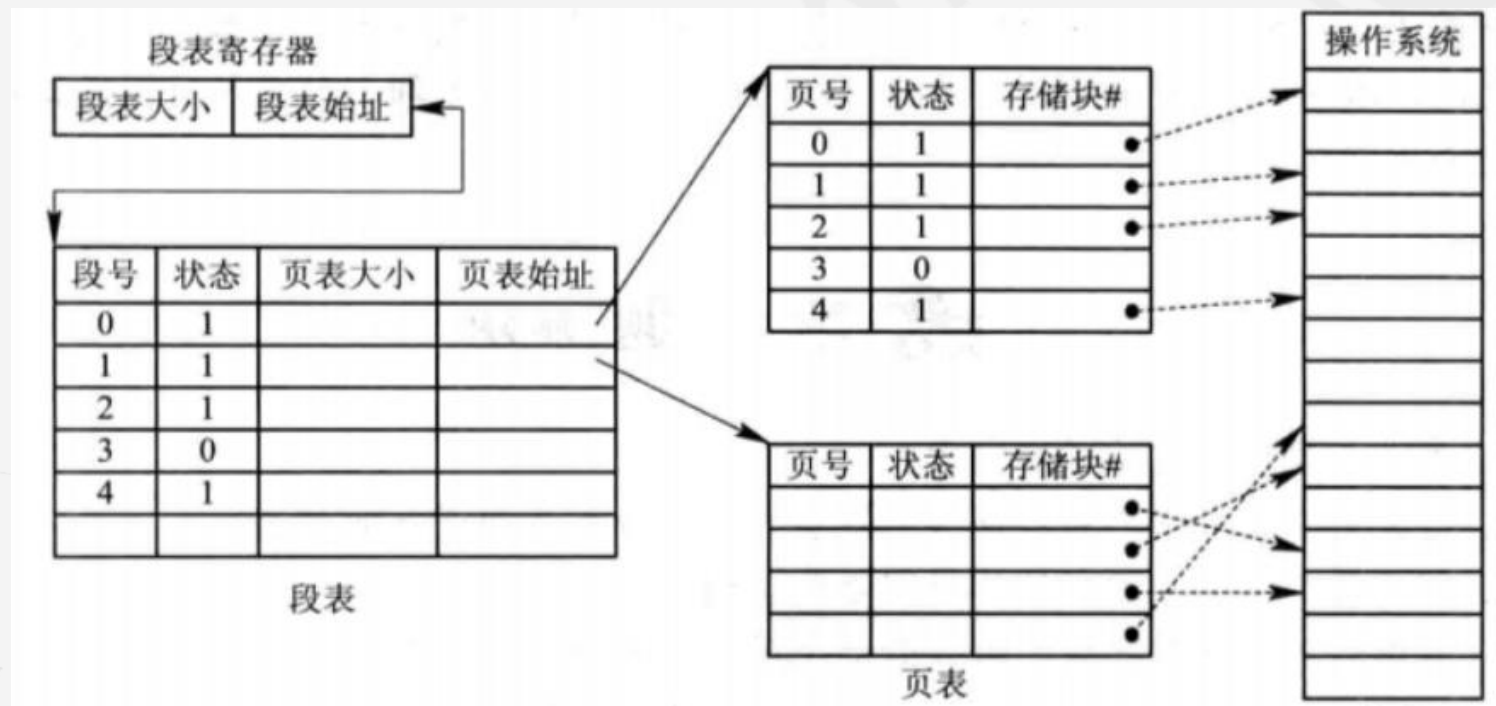
- 分页系统以页面作为内存分配的基本单位，能够有效提高内存的利用率，而分段系统以段作为内存分配的基本单位，它能够更好的满足用户多方面的需求。而段页式系统这很好的集两者之长。段页式系统既有分段系统的便于实现、分段可共享、易于保护、可动态链接等一系列优点；又能像分页系统那样，很好的解决内存的外部碎片问题。
- 段页式系统的基本原理是分段和分页原理的结合，即先将用户程序分为若干段，再把系统分为若干页，为每一个段分配一个段名。
- 段页地址结构包括三部分：段号，段内页号，页内地址。例：





段页式存储管理方式

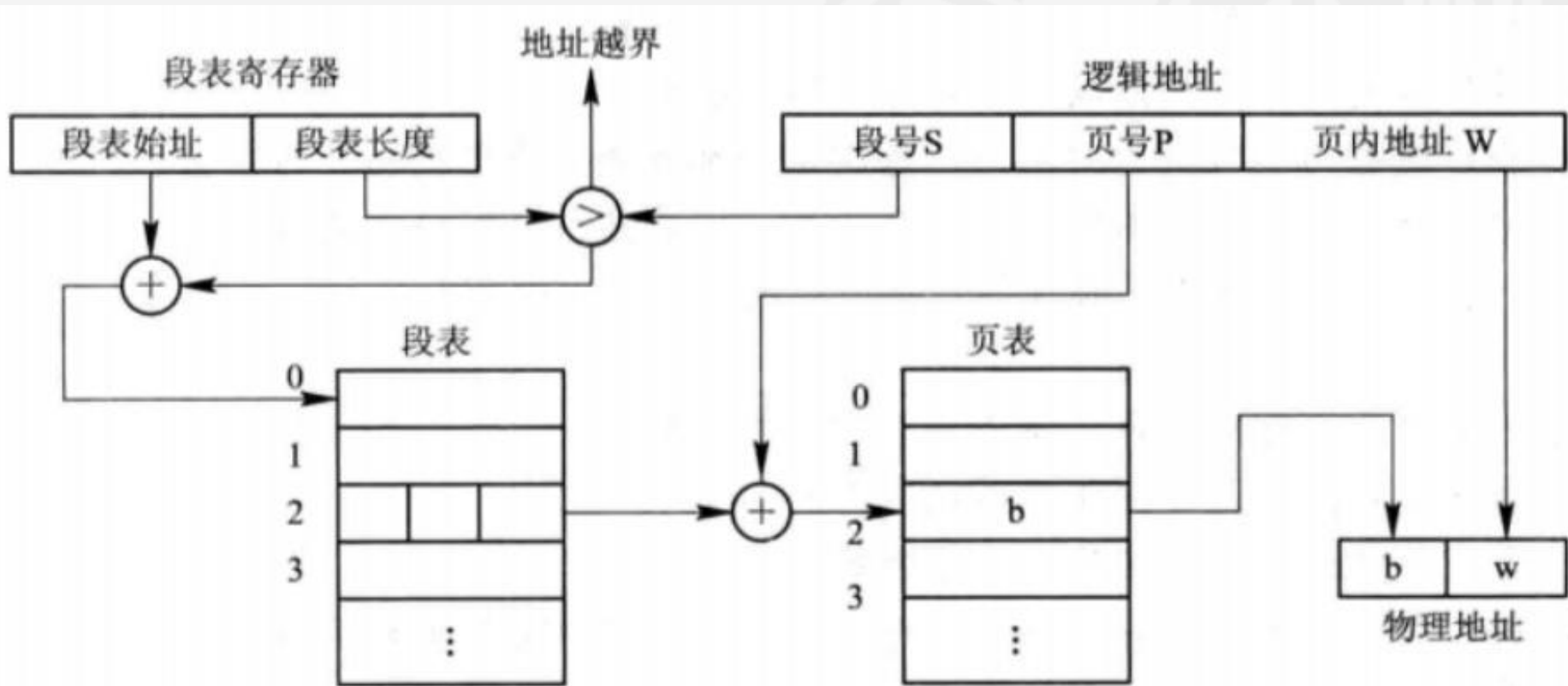
- 段表的内容和分段系统不同，它还包括页表大小和页内始址，下图就是利用段表进行从逻辑地址到物理地址的转变。





段页式地址变换

段页式系统为了便于实现地址变换，须配置一个段表寄存器，其中存放段表始址和段长TL。



进行地址变换时：

1. 将段号S与段长TL进行比较，若 $S < TL$ ，表示未越界。
2. 利用段表始址和段号来求出该段所对应的段表项在段表中的位置，从中得到该段的页表始址。
3. 利用逻辑地址中的段内页号P来获得对应页的页表项位置，从中读出该页所在的物理块号b。
4. 再利用块号b和页内地址来构成物理地址。



段页式与TLB

- 在段页式系统中，为了获取一条指令，需进行三次内存访问，即访问段表，访问页表，访问物理块号。
- 为了提高运行速度，在其中增加一个高速缓冲寄存器。每次访问地址时，现在高速缓存器中查找，如果有，可直接找到物理块。若未找到，在使用原来的三次访问，之后将该物理块存入高速缓存器中。

Comparison field		Page frame	Protection	Age	Is this entry used? ↓
Segment number	Virtual page				
4	1	7	Read/write	13	1
6	0	2	Read only	10	1
12	3	1	Read/write	2	1
					0
2	1	0	Execute only	7	1
2	2	12	Execute only	9	1



各种内存管理方式的比较

	固定分区	可变分区	分页式	分段式	段页式
碎片类型	内部碎片	外部碎片	内部碎片	外部碎片	内部碎片
连续存储的单元	整个进程	整个进程	页	段	页
交换单元	整个进程	整个进程	页	段	段
重定位机构	基址寄存器	基址寄存器	页表	段表	段表和段内页表



课后阅读

- 《操作系统概念 OSC（第9版）》第8章、第9章。
- 《现代操作系统 MOS（第4版）》第3章。
- 《深入理解计算机系统 CSAPP（第3版）》第6章、第9章。